Part 1:

Lab Write-up: Building a Blogging Platform with Django

Objective:

The objective of this lab is to create a robust blogging platform using Django that includes user authentication, permissions, and a RESTful API. Users will be able to create, read, update, and delete their own blog posts, as well as read, like, and comment on other users' posts. Additionally, the lab will include admin functionality with full permissions on all posts.

Prerequisites:

- Basic understanding of Python and Django.
- A development environment set up with Django installed.

Tasks:

Task 1: Database Design

Design the database schema for the blogging platform. Define models for users, blog posts, likes, comments, and user permissions(roles).

Task 2: User Authentication and Teams

Implement user authentication using Django's built-in authentication system. Users should be able to register, log in, and log out. Integrate teams for permissions management.

Task 3: User Permissions

Users can have 1 of 2 roles: “admin” or “blogger”. Each user is a member of exactly 1 team
Admin users can read/edit any post in the website and are not restricted by the permissions system
The permissions system assigns each blog post access controls for reading and writing. The 4 options are:

1. Public: anyone can access the post
2. Authenticated: any authenticated user can access the post

3. Team: Any user on the same team as the post author can access the post
4. Author: Only the author can access the post

The access control for reading and editing are independent of each other. For clarity, each post should have permissions for all 4 levels. Example Post A permissions:
Public - Read Only
Authenticated - Read Only
Team - Read and Edit
Auther - Read and Edit

Task 4: Create a RESTful API

Develop a RESTful API using Django Rest Framework to handle user registration, login, profile management, blog post creation, updates, deletions, likes, comments, and admin access.

Task 5: Like and Comment Functionality

Implement like and comment functionality for users to interact with blog posts via the API. Users can like a blog post and add comments.

Task 6: Admin Panel

Set up Django's admin panel and configure it to manage blog posts and user accounts. Site Admins should be able to perform CRUD operations on all posts. Note that a “site admin” is different than the admin user role.

User Stories:

User Story 1: Blog Post Creation with Permissions

As a developer, I need to provide an API endpoint for blog post creation with configurable permissions for post visibility.

ACs:

1. Endpoint (`/post`) for post creation.
2. Options for setting post permissions for reading and writing
    a. Public: non-logged in users can view the post
    b. Authenticated: Logged in users can view the post, regardless of their team affiliation
    c. Team: Only members of the author’s team can view the post
    d. Owner: Only the author him/herself can view the post
3. Edit and read permissions are independent of each other
4. Author automatically set as the logged-in user
5. Store post with permissions settings in the database

6. Posts have the following attributes:
    a. Author
    b. Title
    c. Content
    d. Excerpt (first 200 characters of a post)
    e. Timestamp

User Story 2: Blog Post Editing and Permission Modification

As a developer, I need to offer an API endpoint for blog post editing, including the ability to modify permissions.

ACs:

1. Endpoint (`/blog/<post_id>`) for editing posts
2. Following fields can be changed
    a. Permissions
    b. Content
    c. Title
3. Allow changes to content and visibility settings
4. Editing follows the permissions set on the post before the attempt to edit the post
5. Save changes in the database.

User Story 3: Viewing and Interacting with Posts Based on Permissions

As a developer, I need to implement viewing and interaction mechanisms for posts, respecting the defined permissions.

ACs:

1. Exists an endpoint (`/post` and `/post/<post_id>`) that allows users to view posts
2. The endpoint follows the permissions assigned to the user accessing the endpoint
3. The list endpoint will only include posts the user has access to in the list
    a. Will return an empty list if there are no posts the user has access to
4. The details endpoint will return a 404 if the user does not have view access to the post
5. Pagination is implemented to limit the number of posts per page to 10.
6. Pagination should include the following information: current page, total pages, total count, next page URL, previous page URL.

User Story 4: Adding Likes to a Blog Post

As a developer, I need to create an API endpoint to allow users to like a blog post, reflecting user engagement with content.

ACs:

1. Exists a mechanism by which users can like a post
2. Prevent multiple likes from the same user on a single post.
3. Any authenticated user with view access can like a post
    a. Non authenticated users cannot like a post regardless of permissions
4. Users can unlike a post that they have previously liked
    a. Must have view access to the post

User Story 5: Viewing likes on a Blog Post

As a developer, I need to create an API endpoint to list likes on a blog post.

ACs:

1. Exists an endpoint that returns a list of likes
2. Possible to filter those likes by blog post and user
3. Users can view all likes on any post for which they have view access
4. Pagination is implemented to limit the number of likes per page to 20.
5. Pagination should include the following information: current page, total pages, total count, next page URL, previous page URL.

User Story 6: Commenting on a Blog Post

As a developer, I need to provide an API endpoint for users to comment on blog posts, facilitating discussions and feedback.

ACs:

1. Exists a mechanism by which users can comment on a post
2. Require authentication for commenting.
3. Any authenticated user with view access can comment on a post
    a. Non authenticated users cannot comment on a post regardless of permissions
4. Users can delete a comment that they have previously posted
    a. Must have view access to the post

User Story 7: Viewing Comments on a Blog Post

As a developer, I need to create an API endpoint to list comments on a blog post, making user discussions visible.

ACs:

1. Exists an endpoint that returns a list of comments
2. Possible to filter those comments by blog post and user
3. Users can view all comments on any post for which they have view access
4. Include commenter's username and comment timestamp.
5. Pagination is implemented to limit the number of commnets per page to 10.
6. Pagination should include the following information: current page, total pages, total count, next page URL, previous page URL.

User Story 8: Deleting a Blog Post

As a developer, I need to implement an API endpoint for the deletion of blog posts, allowing users to remove their content from the platform.

ACs:

1. Possible to delete a blog post
2. Only a user with edit permission can delete a blog post
3. Upon deletion, the post is permanently removed from the database
4. All associated likes and comments are also deleted.

Submission:

- To start your project, you can check out this public branch: https://github.com/samah-m8/etapa7.git
- Submit the Django project source code, including the models, views, serializers, and URL configurations for the API.
- Provide clear API documentation on how to use and interact with the API.
- All code and documentation to be uploaded to a GitHub Repository
- Share your repo with samah-m8 and kevin29a users on GitHub

Evaluation:

Your project will be evaluated based on the following criteria:
- Proper database design with the defined models.
- Correct implementation of user authentication and permissions.
- Functionality for creating, updating, and deleting blog posts via the API.
- Implementation of like and comment functionality via the API.
- Proper configuration of the admin panel with admin permissions.
- Code quality, organization, and adherence to Django and Django Rest Framework best practices.

Part 2:

Lab Write-up: Building a Blogging Platform with Angular

To complement your Django backend for the blogging platform, we will create an Angular frontend application. The Angular app will connect to your Django REST API, providing a user-friendly interface for users to interact with the blog. Here's a detailed plan for the Angular frontend, including groomed user stories and corresponding wireframes.

Frontend Angular Application: Blogging Platform

Objective:
Develop an Angular frontend application that interfaces seamlessly with the Django REST API backend. The application will provide a smooth user experience for creating, viewing, editing, and interacting with blog posts.

Prerequisites:
Basic understanding of TypeScript and Angular.
Angular CLI installed in the development environment.
Access to the Django REST API endpoints.

User Story 1: User Registration

Title: User Registration for Blogging Platform

As a new user, I want to be able to register for an account on the blogging platform so that I can create, like, and comment on blog posts, as well as access personalized features.

Acceptance Criteria:

1. **Registration Page Access**: When a non-registered user visits the platform, they should have an option to navigate to the registration page.

2. **Registration Form**: The registration page should have a form with the following fields:
   - Username (email address) (required)
   - Password (required)
   - Confirm Password (required)

2.1 **Team**: A default team is assigned to new users.

3. **Form Validation**:

- All fields should be validated for proper input (e.g., valid email format, password match).
- Display appropriate error messages for invalid inputs.

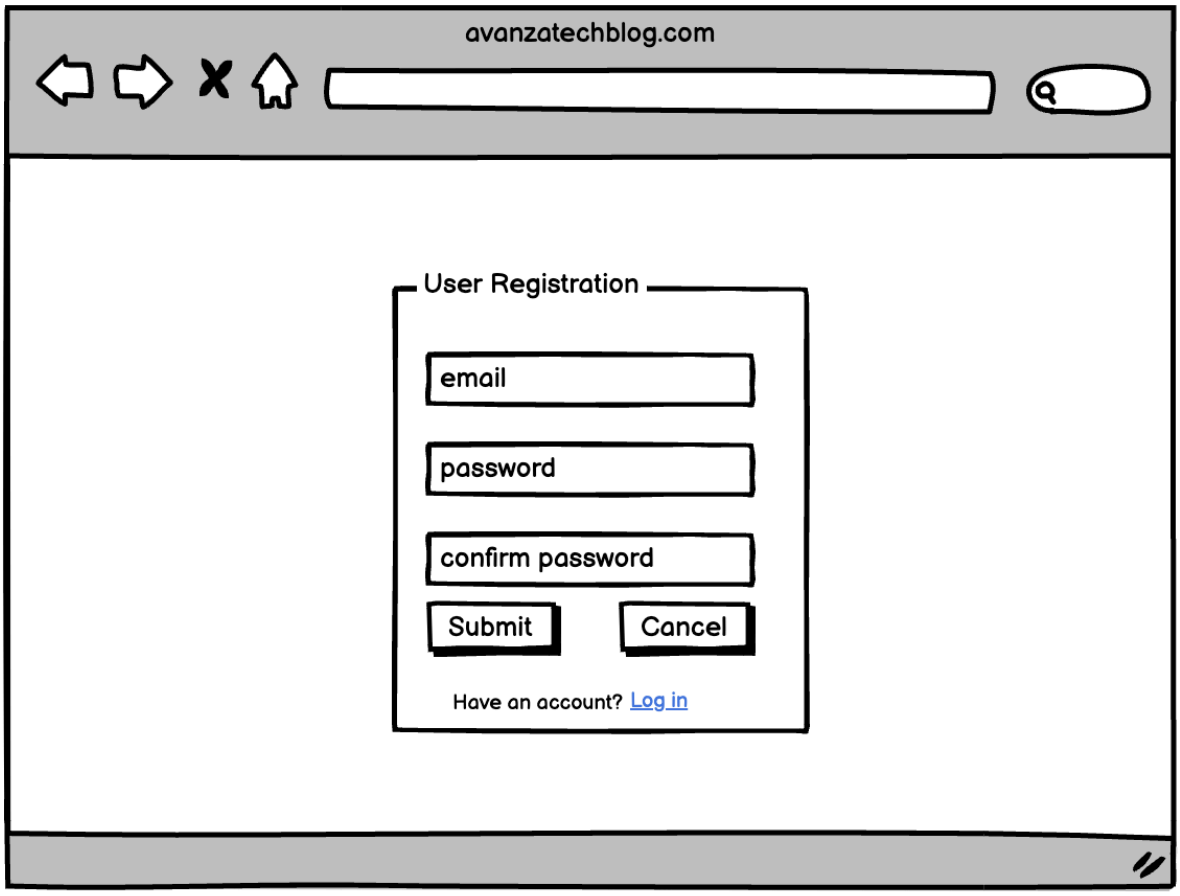
4. **API Integration**: On form submission, send a registration request to the Django backend's registration endpoint. Handle any errors and display appropriate messages.

5. **Success Response**: Upon successful registration, redirect the user to the login page with a success message.

6. **Link to Login**: Include a link for users who already have an account to navigate to the login page.

Notes:

- Ensure the registration process is user-friendly and provides clear feedback on the status of the registration attempt.
- The frontend should handle and display backend validation errors effectively.



User Story 2: User Login

Title: User Login for Blogging Platform

As a user, I want to be able to log in to my account on the blogging platform so that I can access my personalized settings and features such as creating and interacting with blog posts.

Acceptance Criteria:

1. **Login Page Access**: Users should have an option on the homepage to navigate to the login page.

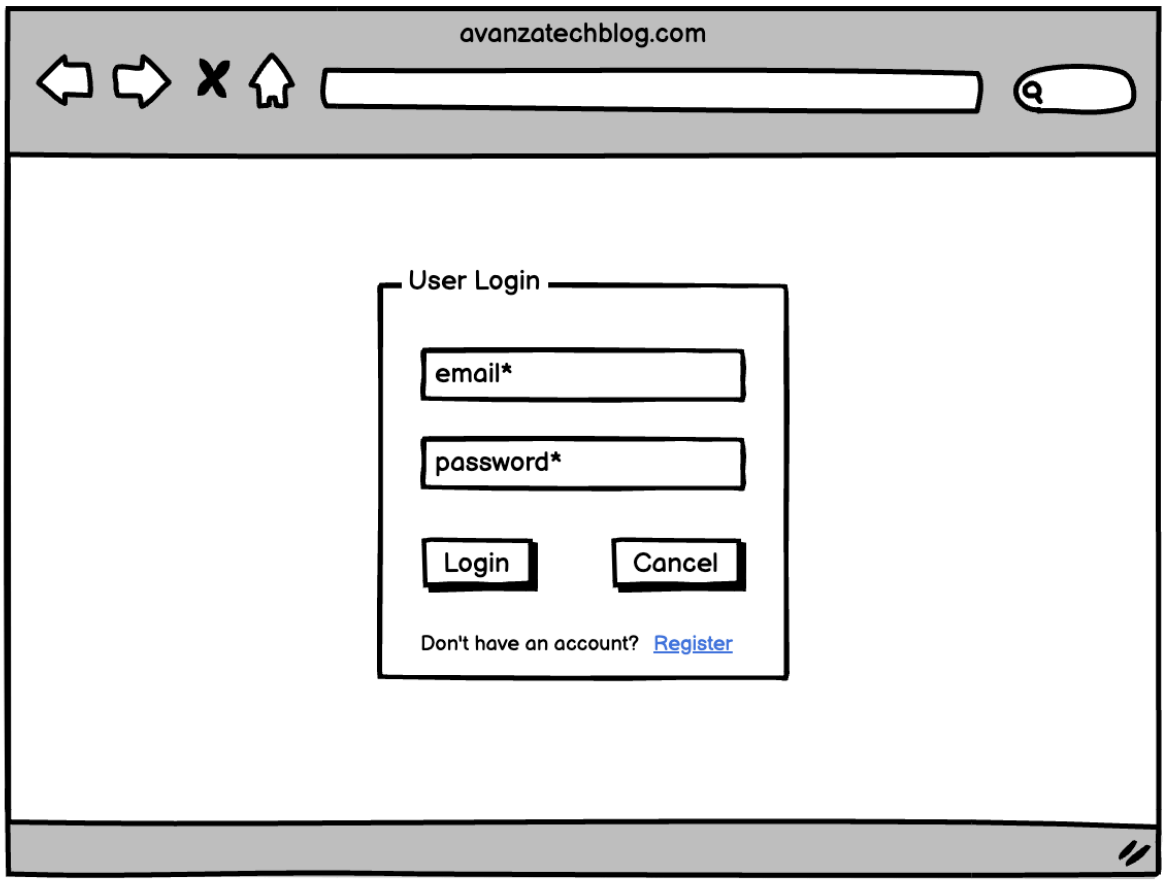
2. **Login Form**: The login page should have a form with the following fields:
   - Username/Email (required)
   - Password (required)

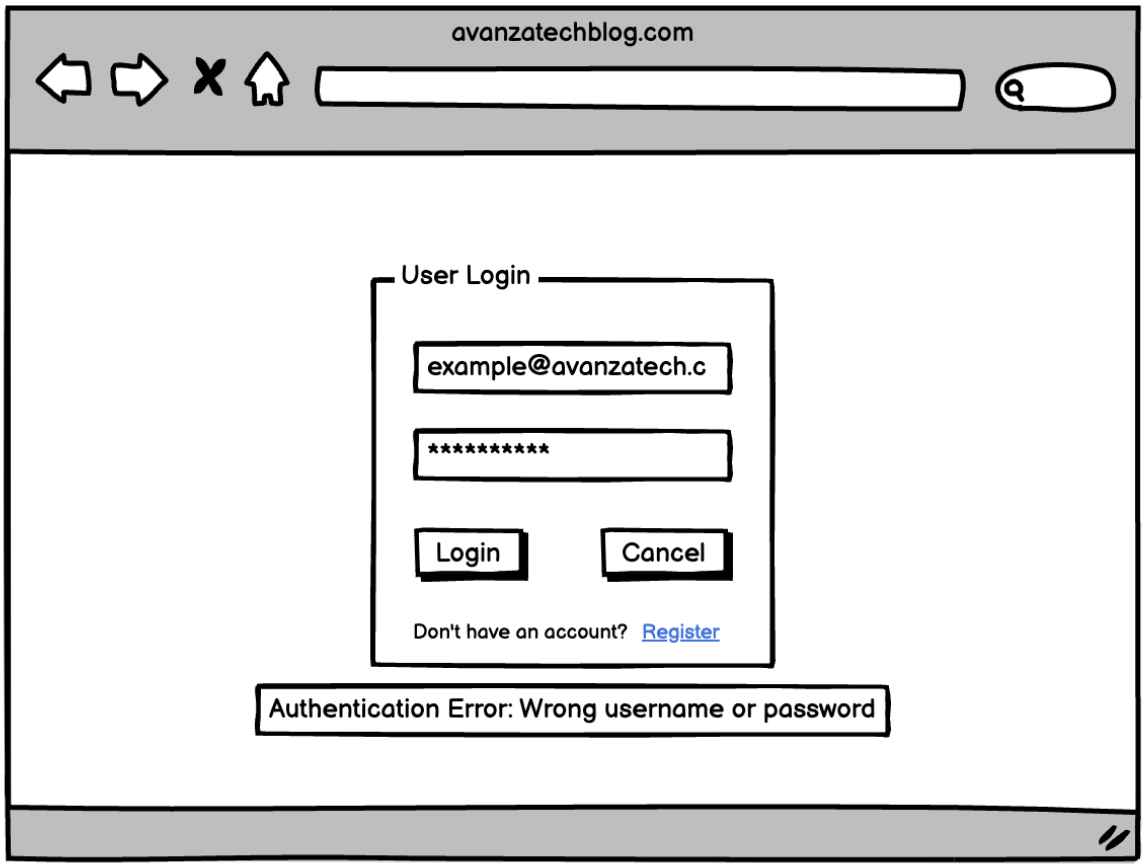
3. **Form Validation**:
   - Validate inputs and display error messages for incorrect data.
   - Include a feature to toggle password visibility.

4. **API Integration**: On form submission, send a login request to the Django backend's login endpoint. Handle any errors and display appropriate messages.

5. **Success Response**: Upon successful login, redirect the user to the homepage, where posts the user can view are listed.

6. **Link to Registration**: Include a link for new users to navigate to the registration page.


avanzatechblog.com
User Login
email*
password*
Login
Cancel
Don't have an account? Register



User Story 3: Home Page Post Viewing and Interaction

Title: Viewing and Interacting with Blog Posts on the Home Page

As a user,I want to view all blog posts that I have access to on the home page, so that I can interact with them through likes and comments, and have the ability to create a new post.

Acceptance Criteria:

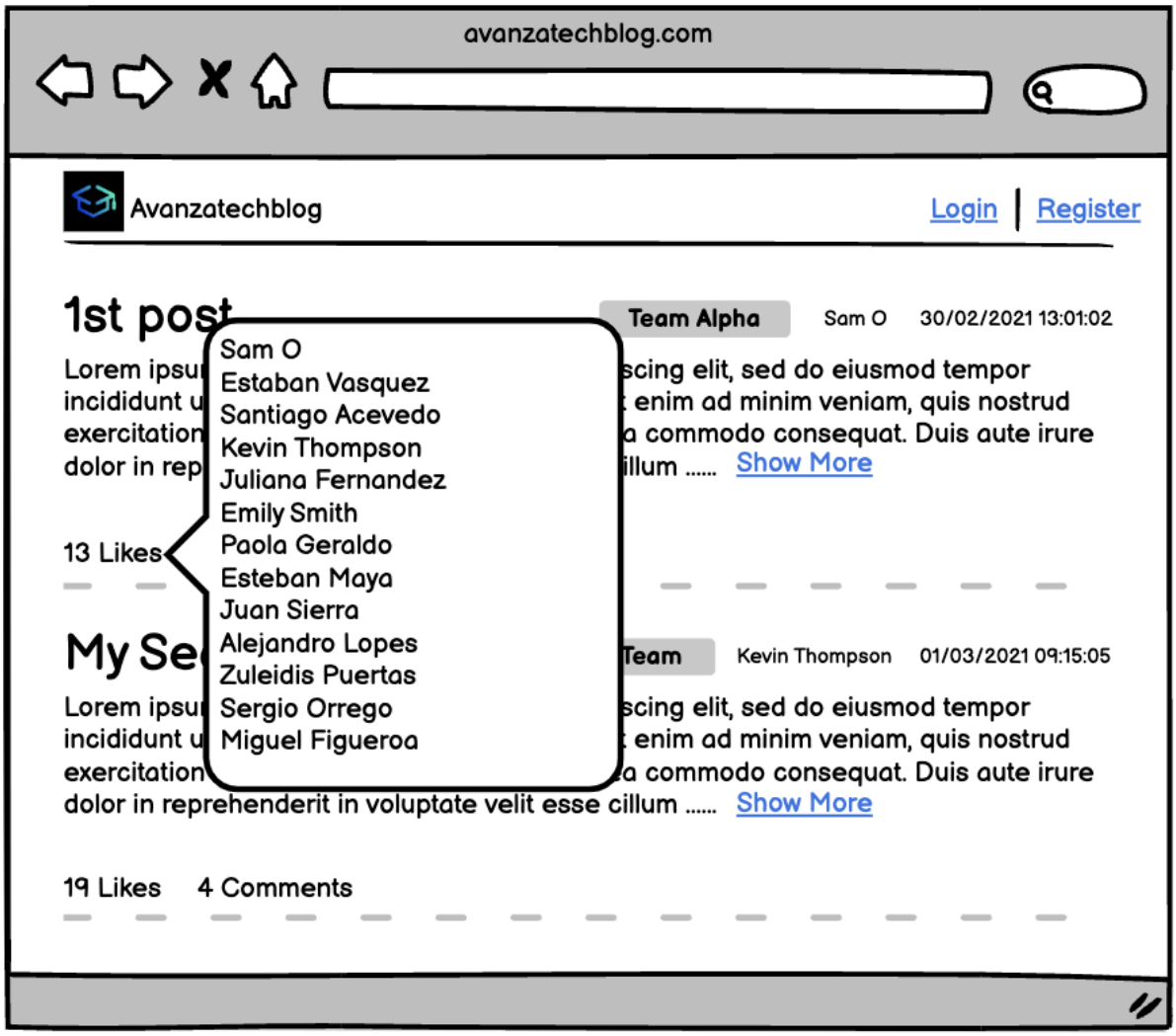
1. **Access to Home Page**: Upon visiting the page, the user is directed to the home page where they can see a list of blog posts.

2. **Viewing Posts**:
   - Users can display all blog posts that the user has permission to view.

- Each post should display the title, an excerpt of the content, the author's team, name, and the date of posting.
- Posts should be displayed in reverse chronological order, with the most recent posts at the top.

3. **Post Interaction**:
- For non-logged in users, posts have no interaction options (Like / Comment).
- Each post should show the title, author’s team, name and creation date and time

- Show the current number of likes on each post.
    - Clicking on the likes counter shows a list of who liked the post
    - List of likes is ordered by like created time.



- Display a count of existing comments on each post.
    - Clicking on the Comments counter redirects to the post details page (independent story).
- Excerpts of the posts are shown, if the post content is longer than the excerpt, a Show More link is shown at the end of the excerpt.
    - Clicking on “Show More” redirects to the post details page (independent story).

- For logged in users, each post should have a 'Like' button. If a post is already liked by the user, the button should reflect that state.
- For logged in users, include a 'Comment' button that navigates to a detailed post view where the user can submit a comment.
- If a user has edit permission on a certain post, an Edit button is shown next to the post title
- Clicking on the Edit button loads the post in an edit form (independent story)
- If a user has edit permission on a certain post, a Delete button is shown next to the Edit button

- Clicking on the Delete button asks for a confirmation pop-up with two buttons: Delete and Cancel
- Clicking Delete deletes the post and redirects to the homepage.
- Clicking cancel closes the pop-up

4. **Creating a New Post**:
   - Provide a prominent 'Create New Post' icon on the home page.

- Clicking on this button navigates the user to a form where they can write and publish a new blog post.
- Creation form is in another story.

5. **Pagination**:
   - Implement pagination to limit the number of posts displayed at one time.
   - Provide navigation options to move between pages of posts, comments and Likes.
   - 10 posts per page, 5 comments per details page, 15 likes per page.

1st post

Team Alpha | Sam O | 30/02/2021 13:01:02

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum ..... Show More

13 Likes 2 Comments

My Second Post

Beta Team | Kevin Thompson | 01/03/2021 09:15:05

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum ..... Show More

19 Likes 4 Comments

1-10 of 23 >

13 Likes 2 Comments

My Second Post

Beta Team | Kevin Thompson | 01/03/2021 09:15:05

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum ..... Show More

19 Likes 4 Comments

< 11-20 of 23 >

avanzatechblog.com

Avanzatechblog

Welcome, Sam

< Back

1st post

Team Alpha Sam O 30/02/2021 13:01:02

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

13 Likes

Comments

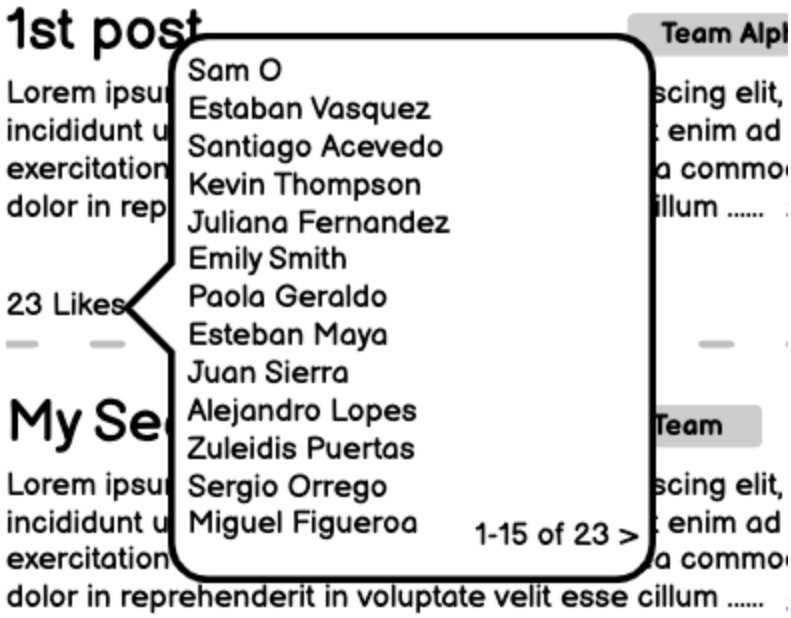
Miguel F. 30/02/2021 14:11:22

Good post, but I'm sick of following my dreams, man. I'm just going to ask where they're going and hook up with 'em later.

Juliana Fernandez 30/02/2021 16:15:41

Gentlemen, you can't fight in here. This is the war room.

1-5 of 7 >



6. **Error Handling**:
   - Gracefully handle any errors that occur while fetching posts (e.g., display a friendly error message).

User Story 4: Post Detail Page

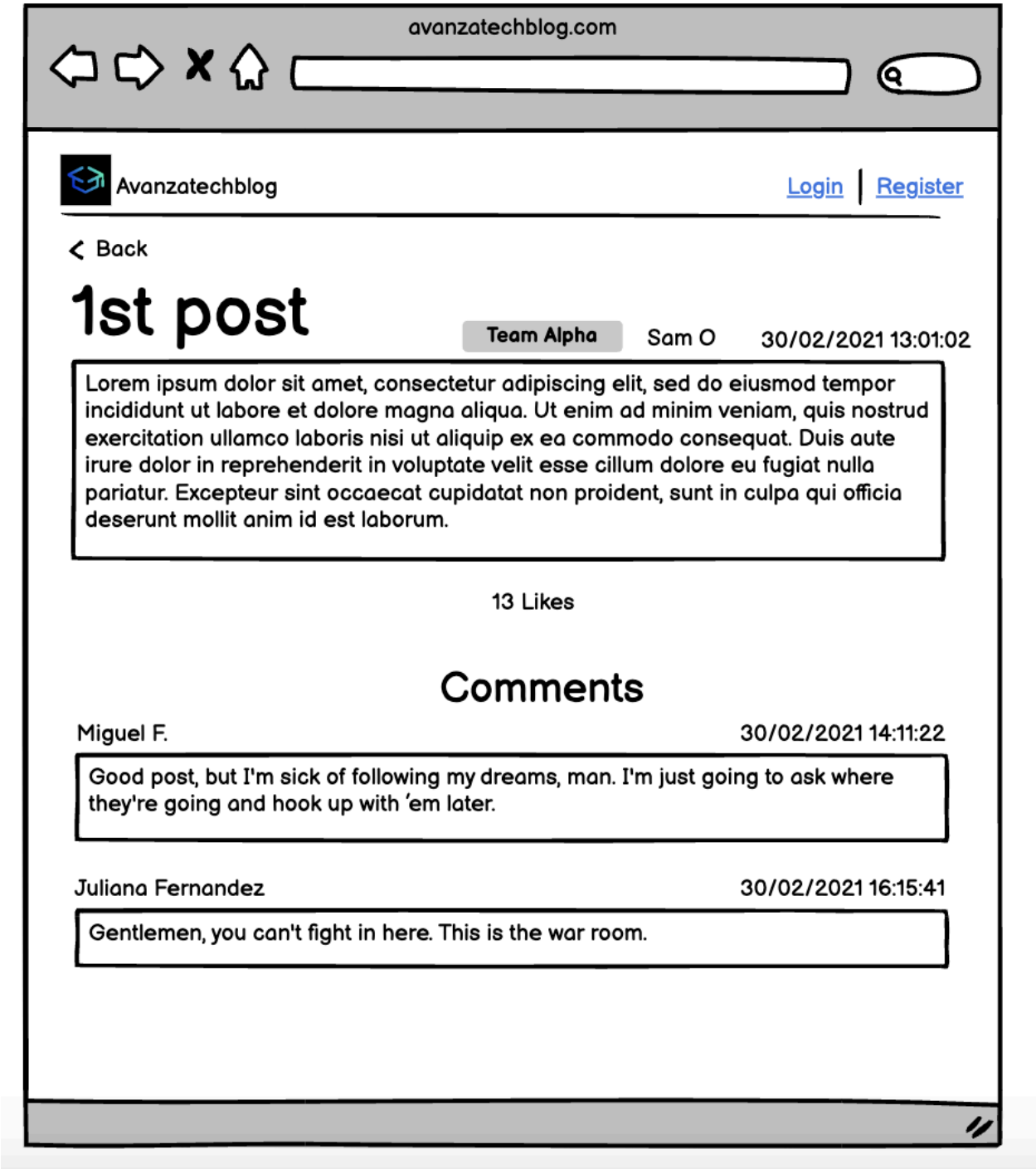
Title: Viewing the details of a post

As a user,I want to view details of a posts that I have access to on the home page, so that I can see the complete post content, read all the its comments, and be able to add my own comments to the post.

Acceptance Criteria:

1. **Access the detail page**: Clicking on the comments counter or the Comment icon in the home page directs to the Post detail page.

2. **Viewing Posts**:
   - Comments are ordered by their creation date, with the oldest post first.
   - Non-logged in users can view the full post and its comments as well as a counter of the likes on the post.
   - Each comment should show its author, creation timestamp and the full comment text



- Logged in users can view the full post and its comments, a counter of the likes on the post, and can write their own comment to the post.

avanzatechblog.com

Avanzatechblog

Welcome, Sam

< Back

1st post

Team Alpha | Sam O | 30/02/2021 13:01:02

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

13 Likes

Comments

Miguel F. 30/02/2021 14:11:22

Good post, but I'm sick of following my dreams, man. I'm just going to ask where they're going and hook up with 'em later.

Juliana Fernandez 30/02/2021 16:15:41

Gentlemen, you can't fight in here. This is the war room.

Add New Comment:

New Comment

Submit Cancel

3. **Redirect After Submitting**:

- After submitting a comment, it is added to the comments list and the page is refreshed to the post detail page.
- Newlines are rendered as expected in a comment after submission.

4. **Cancel Option**:
- Provide a 'Cancel' button to allow users to back out of creating a new comment at any time, resetting the new comment form to default values.

User Story 5: Blog Post Creation

Title: Creating a New Blog Post

As a user, I want to create a new blog post easily from the blogging platform's interface so that I can share my thoughts, ideas, and stories with others.

Acceptance Criteria:

1. **Access to Post Creation Form**:
- Ensure that logged-in users can access the post creation form from the home page via a 'Create New Post' button.

2. **Post Creation Form**:
- Present a form with the following fields:
  - Title (required)
  - Content (required)
  - Categories (dropdown with options: Public, Authenticated, Team , Owner)
  - Permissions: None, Read Only, Read and Write

Default values for permissions:
- Public : read only
- Authorized: read only
- Team: read & write
- Owner: read & write

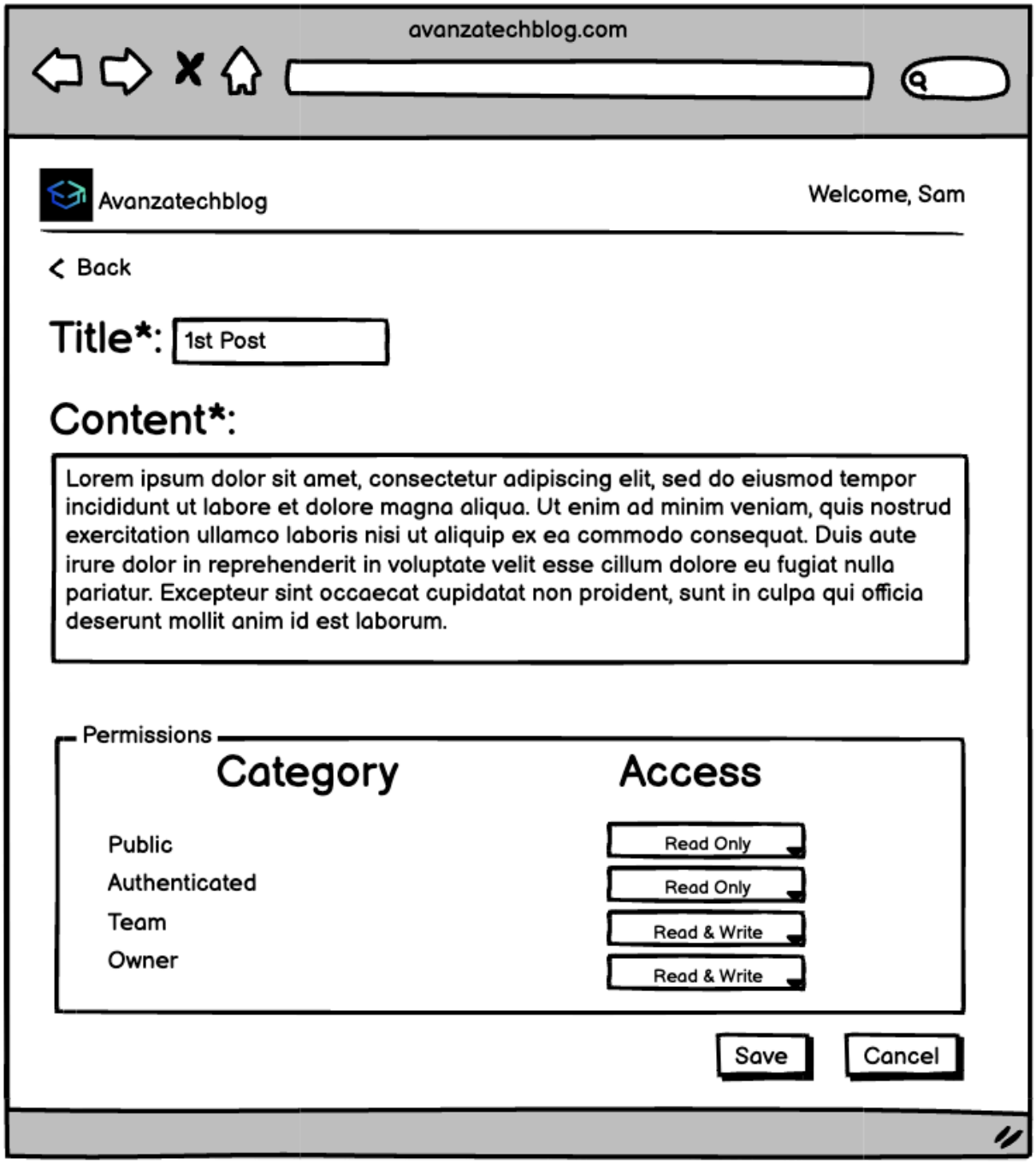
3. **Form Validation**:
- Implement form validation to ensure that all required fields are filled out before submission.
- Provide helpful error messages for invalid inputs or fields left blank.

4. **Saving the Post**:
- Include a 'Publish' button to save the post to the database via the backend API.

- Display a confirmation message upon successful post creation.

5. **Error Handling**:
   - Handle and display any errors from the backend (e.g., network issues, server errors) in a user-friendly manner.

8. **Redirect After Posting**:
   - After a successful post creation, redirect the user to the home page.
9. **Cancel Option**:
   - Provide a 'Cancel' button to allow users to back out of creating a new post at any time, redirecting them to the previous page they were on.



User Story 6: Editing a Blog Post

Title: Editing an Existing Blog Post

As a userI want to be able to edit my previously published blog posts so that I can update the content, correct mistakes, or change the visibility permissions.

Acceptance Criteria:

1. **Access to Edit Function**:
   - Bloggers should have an 'Edit' option on their own blog posts, visible when viewing the post in detail.

2. **Edit Page/Form**:
   - Navigating to the edit option loads a form pre-populated with the existing post's details, including title and content.
   - The form layout and options should mirror the post creation form for consistency.

3. **Rich Text Editing**:
   - The content field should include a rich text editor to allow for easy formatting changes.

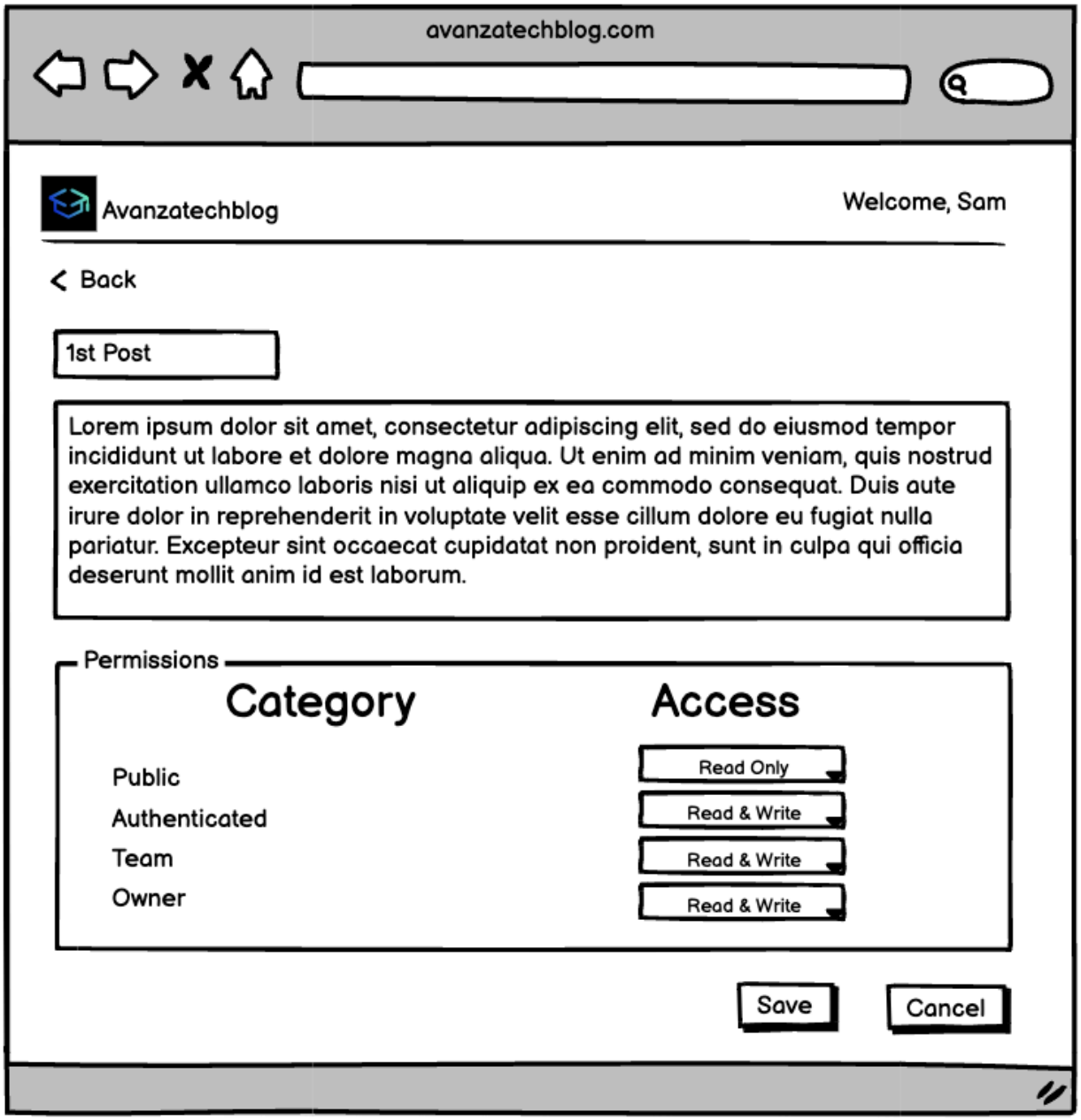
4. **Permission Modification**:
   - Include two dropdown menus to change the post's permissions settings, allowing the blogger to adjust who can view and/or edit the post.
        - Categories: (Public, Authenticated, Team, Owner)
        - Permissions: None, Read Only, Read and Write
5. **Form Validation and Error Handling**:
   - Validate input fields and ensure the form cannot be submitted with mandatory fields empty.
   - Display helpful error messages for validation errors or submission issues.

6. **Saving Changes**:
   - Provide a 'Save Changes' button to submit the updated content and permissions to the backend.
   - Display a confirmation message upon successful update or an error message if the update fails.

7. **Cancel and Navigate Back**:
   - Include a 'Cancel' button to abort editing and return to the previous view without making changes.

8. **Security and Permissions**:
   - Ensure that only the post author or users with appropriate permissions can access the edit function.

9. **Redirect After Editing**:
   - After a successful edit, redirect the user to the updated post's detail view confirming the successful update.



User Story 7: User Logout

Title: Secure Logout from the Blogging Platform

As a user, I want to be able to securely log out of the blogging platform to ensure my account is safe when I'm not using it.

Acceptance Criteria:

1. **Logout Option Accessibility**:
   - Provide a clearly visible and accessible 'Logout' option in the user interface.

2. **Confirmation Prompt**:
   - Upon clicking on the logout button, a confirmation prompt is shown to verify the user's intention to log out.

3. **API Integration**:
   - The logout process should send a request to the backend to properly terminate the session.
   - Ensure that any authentication tokens or session information are invalidated upon logout.

4. **Redirection After Logout**:
   - Upon successful logout, redirect the user to public home page of the platform.
   - Display a confirmation message indicating that the user has been successfully logged out.

5. **Security Considerations**:
   - Ensure the logout process is secure and that the user's session cannot be reused after logout.

6. **User Feedback**:
   - Provide immediate visual feedback when the logout process is initiated, such as a loading indicator or a temporary disablement of the logout button to prevent multiple clicks.

avanzatechblog.com
Avanzatechblog
Welcome, Sam
1st post
Team Alpha
Sam O
30/02/2021 13:01:02
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum ..... Show More
13 Likes
2 Comments